

Kacper

Osoba 3 - Kamera, oświetlenie, tekstury, cele

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/Camera.*, Engine/Light.*, Engine/Texture.*, Demo/Targets.cpp, assets/textures/

0. Gdzie sa moje pliki (mapa do prezentacji)

Strzałka <<< pokazuje pliki, które robiłem ja (Kacper).

```
pgk/
|
|-- Engine/
|   |-- Math3D.* SceneNode.*          (Lukasz)
|   |-- PrimitiveNode.*              (Rogert)
|   |-- Camera.h / Camera.cpp <<< MOJE (kamera FPP i orbita)
|   |-- Light.h / Light.cpp <<< MOJE  (swiatlo + material Phong)
|   |-- Texture.h/ Texture.cpp <<< MOJE (BMP + tekstury proceduralne)
|   |-- Engine.*                     (Jakub)
|
|-- Demo/
|   |-- Room.cpp                     (Lukasz)
|   |-- Obstacles.cpp                (Rogert)
|   |-- Targets.cpp <<< MOJE (cele, fale, ruch celow)
|   |-- Gameplay.cpp Constants.h ... (Jakub)
|
+-- assets/textures/*.bmp <<< MOJE (pliki tekstur + instrukcja)
```

W skrocie: ja zajmuje sie tym, jak scena **wyglada** i jak **patrzemy** na nia. Kamera (skad i w ktora strone), oświetlenie (model Phong), tekstury (wczytane z BMP albo wygenerowane wzorami). W grze robie **cele** - ich pojawianie sie, ruch i animacje.

1. O co chodzi w moich plikach (po ludzku)

Kamera decyduje, co widzimy. Mam dwa tryby: 'orbita' (kamera krazy wokół punktu) i 'pierwsza osoba' (FPP - oko stoi w miejscu, a my rozglądamy się myszka). W grze używamy FPP.

Światło i **material** razem dają realistyczne cieniowanie (model Phong: ambient + diffuse + specular).

Tekstury to obrazki nalepiane na obiekty - albo wczytane z pliku BMP (sam napisałem parser), albo wygenerowane algorytmem (szachownica, drewno, cegły, marmur, szum).

W **Targets.cpp** robię cele do strzelania: losowe pojawianie się, ruch tam i z powrotem, obrót i kołysanie.

2. Camera - dwa tryby patrzenia

Kamera musi wyprodukować **macierz widoku** - mówi OpenGL, gdzie jest oko i w którą stronę patrzy. W trybie FPP liczę kierunek patrzenia z dwóch kątów: **yaw** (obrot lewo-prawo) i **pitch** (gora-dół).

Camera.cpp - kierunek patrzenia i macierz widoku

```
Vec3 Camera::lookDirection() const {
    if (cameraMode == Mode::FIRST_PERSON) {
        return Vec3( sin(yaw)*cos(pitch),    // X
                    sin(pitch),              // Y (gora/dol)
                    -cos(yaw)*cos(pitch) ); // Z (do przodu = -Z)
    }
    return normalize(orbitTarget - eyePosition()); // tryb orbity
}

Mat4 Camera::viewMatrix() const {
    Vec3 eye = eyePosition();
    Vec3 center = (cameraMode == Mode::FIRST_PERSON)
        ? (eye + lookDirection()) // FPP: patrz przed siebie
        : orbitTarget;           // orbita: patrz na cel
    return Mat4::lookAt(eye, center, Vec3(0,1,0));
}
```

Funkcję lookAt napisał Lukasz - ja jej używam, podając pozycję oka i punkt, na który patrzymy.

Ograniczam pitch do +/- 85 stopni, żeby nie dało się 'przewrócić' kamery do góry nogami.

3. Light - oświetlenie Phong

Korzystam z oświetlenia OpenGL (GL_LIGHT0..7). Material ma 3 składniki:

- **ambient** - kolor w cieniu (światło otoczenia, zawsze obecne),
- **diffuse** - główny kolor obiektu pod światłem rozproszonym,
- **specular** - błysk/odbicie, plus **shininess** mówi, jak ostry jest błysk.

Light.cpp - ustawienie światła w danej pozycji sceny

```
void PointLight::renderSelf(const Mat4& worldMatrix) const {
    GLenum lightId = GL_LIGHT0 + activeLightIndex;
    if (!isEnabled) { glDisable(lightId); return; }

    glPushMatrix();
    glMultMatrixf(worldMatrix.data());
    GLfloat position[4] = { 0,0,0, 1.0f }; // 1.0 = światło punktowe
    glEnable(lightId);
    glLightfv(lightId, GL_POSITION, position);
    glLightfv(lightId, GL_DIFFUSE, diffuse);
    // ... ambient, specular, attenuation ...
    glPopMatrix();
}
```

Pozycja światła to (0,0,0) **po** nałożeniu macierzy świata wezła - czyli światło świeci z miejsca, gdzie stoi jego wezeł. Czwarta współrzędna = 1 oznacza światło punktowe (gdyby 0, byłoby kierunkowe jak słońce).

Attenuation to tłumienie - im dalej, tym ciemniej, według wzoru $1/(k_c + k_l \cdot d + k_q \cdot d^2)$.

Ważne: światła muszą być w grafie sceny **przed** geometrią, bo renderRecursive idzie po kolei. Gdyby ściana była przed światłem, nie zostałaby nim oświetlona.

4. Texture - wczytywanie i generowanie obrazkow

4.1. Wlasny parser BMP

Napisałem ręczny loader 24-bitowych BMP. Czyta nagłówki bajt po bajcie, sprawdza format i przerabia piksele. Dwie pułapki BMP:

- piksele są w kolejności **BGR** (nie RGB) - trzeba zamienić miejscami R i B,
- wiersze są zapisane **od dołu do góry** - trzeba je odwrócić,
- każdy wiersz jest **dosztukowany do wielokrotności 4 bajtów** (padding).

Texture.cpp - sedno parsera BMP

```
// konwersja BGR -> RGB z odwróceniem wierszy
for (int row = 0; row < h; ++row) {
    int srcRow = topDown ? row : (h - 1 - row); // odwrócenie
    for (int col = 0; col < w; ++col) {
        int src = srcRow * rowStride + col * 3; // rowStride = padding 4B
        int dst = (row * w + col) * 3;
        pixels[dst+0] = raw[src+2]; // R (z pozycji B)
        pixels[dst+1] = raw[src+1]; // G
        pixels[dst+2] = raw[src+0]; // B (z pozycji R)
    }
}
```

4.2. Tekstury proceduralne (bez plików)

Mam 7 generatorów: szachownica, gradient, paski, szum (Perlin/FGM), drewno, cegły, marmur. To czysta matematyka na pikselach. Przykład - drewno to koncentryczne kręgi zaburzone szumem:

Texture.cpp - generateWood (słoje drewna)

```
float turb = fbm2D(fx*2.5f, fy*2.5f, 4, seed) - 0.5f; // zaburzenie
float dist = sqrt(fx*fx + fy*fy) + turb*0.20f; // odległość od środka
float ring = dist * rings;
float fract = ring - floor(ring); // pozycja w obrębie słoja (0..1)
// jasne tło, ciemne słoje - interpolacja koloru wg fract
```

fbm2D (Fractional Brownian Motion) sumuje kilka 'oktaw' szumu o coraz drobniejszej skali - daje naturalny, samopodobny wzór. To moja własna implementacja value noise (losowe wartości na siatce + płynna interpolacja smoothstep).

4.3. Własne mipmapy

Mipmapy to pomniejszone wersje tekstury (1/2, 1/4, ...). OpenGL używa ich dla obiektów daleko, żeby nie migotały. Generuje je ręcznie - każdy poziom to усреднение bloków 2x2 z poprzedniego.

Texture.cpp - ręczne generowanie mipmap

```
while (mipW > 1 || mipH > 1) {
    int newW = max(1, mipW/2), newH = max(1, mipH/2);
    // dla każdego piksela: średnia z 4 pikseli (2x2) z wyższego poziomu
    glTexImage2D(GL_TEXTURE_2D, level, GL_RGB, newW, newH, 0,
                 GL_RGB, GL_UNSIGNED_BYTE, dst.data());
    mipW = newW; mipH = newH; ++level;
}
```

5. Demo/Targets.cpp - cele do strzelania

Cele to sfery, które pojawiają się w falach po 1-3 sztuki. Trzymam **pule** 3 gotowych sfer i tylko chowam/pokazuje je (setVisible), zamiast tworzyć i niszczyć - to szybsze i prostsze.

5.1. Ruch celu

Targets.cpp - pozycja celu liczona z sinusa (ruch ping-pong)

```
Vec3 ShootingGallery::currentTargetPos(const Target& t) const {
    Vec3 pos = t.basePos;
    if (t.moveRange > 0.01f)    // cel rusza się tam i z powrotem
        pos += t.moveAxis * (t.moveRange * sin(totalTime_*t.moveSpeed + t.movePhase));
    pos.y += 0.18f * sin(t.bobPhase * 2.2f);    // delikatne kołysanie w pionie
    return pos;
}
```

Sinus daje płynny ruch tam i z powrotem. **movePhase** jest losowe, żeby cele nie ruszały się zsynchronizowane. Ta sama funkcja jest używana przy rysowaniu i przy strzale - dzięki temu strzał trafia tam, gdzie cel naprawdę jest w danej chwili.

5.2. Bezpieczne pojawianie

Przy losowaniu pozycji celu sprawdzam: czy nie jest za blisko gracza, czy nie jest w środku przeszkody (używam isInsideObstacle Rogerta, z marginesem na cały zakres ruchu) i czy nie nachodzi na inny cel. Probuje do 60 razy, zanim dam za wygraną.

6. Pytania od wykładowcy - przygotowanie

Kamera

P: Czym różni się tryb orbity od pierwszej osoby?

O: W orbicie oko krąży wokół stałego punktu (target) - dobre do oglądania obiektu. W FPP oko stoi w miejscu, a yaw/pitch obracają kierunek patrzenia - tak jak w grach FPS. W grze używamy FPP.

P: Jak z kątów yaw i pitch liczysz kierunek patrzenia?

O: To współrzędne sferyczne. $X = \sin(\text{yaw}) * \cos(\text{pitch})$, $Y = \sin(\text{pitch})$, $Z = -\cos(\text{yaw}) * \cos(\text{pitch})$. Przy yaw=0, pitch=0 patrzymy w -Z (do przodu w OpenGL). yaw obraca w poziomie, pitch w pionie. $\cos(\text{pitch})$ skaluje składowe poziome, żeby wektor został jednostkowy.

P: Dlaczego ograniczasz pitch do +/- 85 stopni?

O: Żeby nie dało się patrzeć idealnie w gore/dół i 'przekrecić' kamery do góry nogami. Przy 90 stopniach wektor patrzenia stałby się równoległy do osi góry, co psuje lookAt (degeneracja).

P: Co to jest macierz widoku?

O: Macierz, która przenosi cały świat do układu kamery - tak, jakby kamera była w początku układu i patrzyła w -Z. Buduje ją przez lookAt z pozycji oka i punktu, na który patrzymy.

Oświetlenie

P: Wytlumacz model Phong.

O: Kolor punktu to suma trzech składników: ambient (stałe światło otoczenia, widoczne nawet w cieniu), diffuse (zależny od kąta między normalną a kierunkiem światła - rozproszenie) i specular (jasny błysk odbicia, sterowany przez shininess). OpenGL liczy to za nas, my podajemy parametry materiału i światła.

P: Co oznacza czwarta współrzędna pozycji światła (w=1)?

O: w=1 to światło punktowe - ma konkretną pozycję i świeci we wszystkie strony. w=0 to światło kierunkowe (jak słońce) - liczy się tylko kierunek, nie pozycja. My używamy punktowych (w=1).

P: Co to jest attenuation (tłumienie)?

O: Spadek jasności światła z odległością, wg wzoru $1/(k_c + k_l * d + k_q * d^2)$. k_c to stała, k_l liniowa, k_q kwadratowa. Mniejsze współczynniki = światło sięga dalej. Ustawiamy małe, żeby cały pokój był oświetlony.

P: Dlaczego światła muszą być dodane do sceny przed geometrią?

O: Bo renderRecursive przechodzi węzły po kolei i ustawia stan OpenGL na bieżąco. Światło ustawia glLight dopiero gdy do niego dojdziemy. Geometria narysowana wcześniej nie 'widzi' tego światła. Dlatego dodaje światła jako pierwsze dzieci roota.

Tekstury

P: Dlaczego w BMP zamieniasz R z B?

O: Bo BMP zapisuje piksele w kolejności BGR (niebieski, zielony, czerwony), a OpenGL chce RGB. Więc przy kopiowaniu bierze bajt B na pozycję R i odwrotnie.

P: Czemu odwracasz wiersze obrazu?

O: BMP z dodatnią wysokością trzyma wiersze od dołu do góry (pierwszy wiersz pliku to dół obrazu). OpenGL oczekuje od góry. Więc $\text{wiersz srcRow} = h - 1 - \text{row}$, czyli czytam od końca.

P: Co to jest rowStride / padding w BMP?

O: Każdy wiersz BMP jest dopełniany zerami do wielokrotności 4 bajtów. $\text{rowStride} = (w * 3 + 3) \& \sim 3$ to faktyczna długość wiersza w pliku. Bez uwzględnienia tego obraz by się 'przesuwał' skosnie.

P: Co to są mipmapy i po co je liczysz?

O: To pomniejszone kopie tekstury (1/2, 1/4...). Gdy obiekt jest daleko, OpenGL używa mniejszej wersji - inaczej tekstura migocze i szumi. Liczę je ręcznie: każdy poziom to średnia z bloków 2x2 poprzedniego poziomu.

P: Czym jest FBM / value noise w teksturach proceduralnych?

O: Value noise to losowe wartości na siatce, płynnie interpolowane (smoothstep). FBM sumuje kilka takich szumów o coraz drobniejszej skali i mniejszej sile - daje naturalny, 'chmurkowy' wzór. Używam go w drewnie, marmurze, ceglach i suficie.

Targets

P: Po co pula celów zamiast tworzenia sfer na bieżąco?

O: Tworzenie i niszczenie obiektów OpenGL co chwile jest kosztowne i ryzykowne. Lepiej raz stworzyć 3 sfery i tylko je chować (`setVisible false`) i pokazywać. To wzorzec 'object pool'.

P: Dlaczego ruch celu liczysz sinusem?

O: Sinus daje gładki ruch tam i z powrotem (ping-pong) bez skoków na końcach. $\text{pos} = \text{base} + \text{os} * \text{zakres} * \sin(\text{czas} * \text{predkosc} + \text{faza})$. Faza jest losowa, żeby cele nie ruszały się równocześnie.

P: Czemu ta sama funkcja liczy pozycje przy rysowaniu i przy strzale?

O: Żeby strzał trafiał dokładnie tam, gdzie cel jest w danej klatce. Gdybym liczył pozycję inaczej do rysowania, a inaczej do kolizji, celowanie byłoby niesprawiedliwe - widziałbym cel gdzie indziej niż jest 'naprawdę'.

P: Jak unikasz pojawienia się celu w ścianie albo w przeszkodzie?

O: Losuje pozycję i sprawdzam warunki: dystans od gracza, czy nie w przeszkodzie (z marginesem na cały zakres ruchu), czy nie za blisko innego celu. Jeżeli warunek niespełniony, losuje ponownie - do 60 prób. Margines uwzględnia ruch, więc cel nie wjedzie w kolumnę podczas oscylacji.